Abstract:

The Bellman-Ford algorithm is a classic method for determining the shortest paths from a single source vertex to all other vertices in a weighted graph. Unlike Dijkstra's algorithm, it can handle negative edge weights, enhancing its versatility in practical applications. The algorithm works by repeatedly relaxing all edges and updating the shortest known distances until optimal paths are identified. Furthermore, Bellman-Ford can detect negative weight cycles, making it valuable for network routing and optimization problems where such conditions may arise.

Introduction:

To find the shortest path from a single source vertex to all other vertices in a weighted graph, it was developed by Richard Bellman and Lester Ford in the 1950s. Unlike Dijkstra's algorithm, Bellman-Ford can handle graphs with negative edge weights, which makes it more flexible for a wider range of applications. The algorithm works by repeatedly relaxing all edges in the graph, gradually reducing the estimated distances between vertices until the shortest paths are found. Although it has a higher time complexity compared to other shortest path algorithms, Bellman-Ford is especially useful because it can also detect negative weight cycles, which are paths that reduce the total cost indefinitely—a feature that Dijkstra's algorithm cannot handle.

Advantages of the Bellman-Ford Algorithm:

- The Bellman-Ford algorithm can handle negative edge weights, making it useful in applications involving negative costs.
- It can detect negative weight cycles, which helps in determining situations that could lead to infinite results.
- The algorithm is relatively straightforward to understand and implement compared to some other algorithms.

Disadvantages of the Bellman-Ford Algorithm:

- The algorithm has a higher time complexity (O(VE)), making it slower than Dijkstra's algorithm in graphs with positive weights only.
- It requires multiple passes through all edges, which increases execution time.
- While flexible, it may not always be the best choice, especially in cases where graphs only contain positive weights.

Analysis:

The Bellman-Ford algorithm is used to find the shortest path from a single starting point to all other points in a graph. It operates by iterating through all the edges multiple times to ensure that the shortest paths are accurately determined. While it is slower than Dijkstra’s algorithm, it has a significant advantage: it can handle negative edge weights and even detect negative cycles, a capability that Dijkstra cannot provide.

This makes Bellman-Ford particularly useful in scenarios such as network routing or any problem where the cost between two points may be negative. For example, it can be applied in financial networks where costs can fluctuate, allowing for more accurate cost assessments.

Pseudo code:

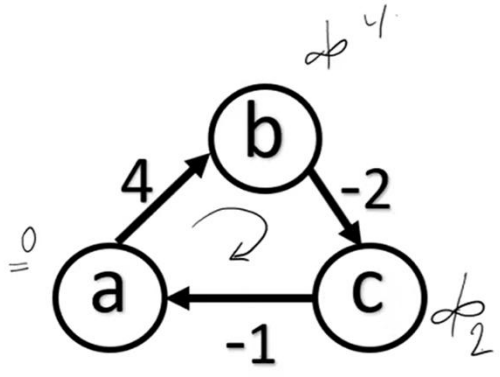
Bellman-Ford Algorithm

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

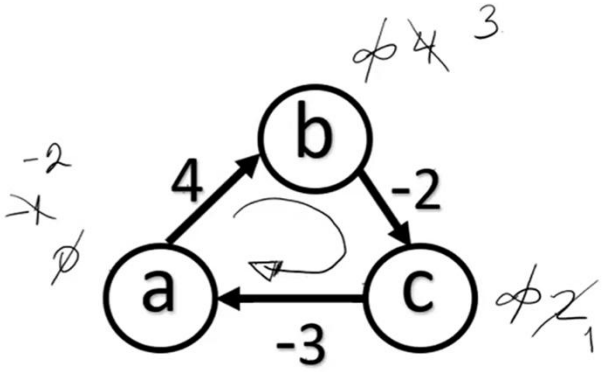
14

Negative cycle:

Bellman-Ford Algorithm

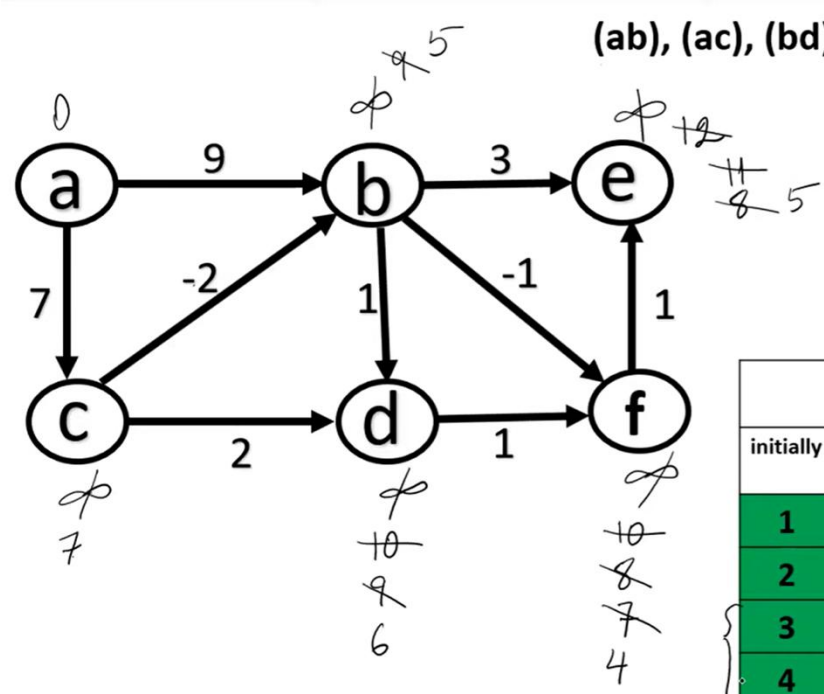


$\sum(4-2-1) = 1$



$\sum 4-2-3 = -1$

Implementation:



	a	b	c	d	e	f
initially	0	∞	∞	∞	∞	∞
1	0	5	7	9	11	8
2	0	5	7	6	8	4
3	0	5	7	6	5	4
4	0	5	7	6	5	4
5						

Appendix:

```
Output - BellmanFord (run)
run:
Vertex  Distance from a
a               0
b               5
c               7
d               6
e               5
f               4
BUILD SUCCESSFUL (total time: 0 seconds)
```

References:

1. R. Nascimento de Cos, "Negative Cycle Detection Using Bellman-Ford and Its Correctness," *Computer Science Stack Exchange*, Jun. 9, 2020. [Online]. Available: https://cs.stackexchange.com/questions/126945/negative-cycle-detection-using-bellman-ford-and-its-correctness. [Accessed: Nov. 1, 2025].

2. D. T. Ahire, "Bellman-Ford Implementation and Simultaneously Relaxation," *Stack Overflow*, Dec. 6, 2020. [Online]. Available: https://stackoverflow.com/questions/65166477/bellman-ford-implementation-and-simultaneously-relaxation. [Accessed: Nov. 1, 2025].

3. "Glossary | Algo: Bellman-Ford Algorithm," *Rosalind*. [Online]. Available: https://rosalind.info/glossary/algo-bellman-ford-algorithm. [Accessed: Nov. 1, 2025].

4. "Bellman-Ford Algorithm | Dynamic Programming," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23. [Accessed: Nov. 1, 2025].

5. "Bellman–Ford Algorithm," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Bellman–Ford_algorithm. [Accessed: Nov. 1, 2025].